Optimization-based Control (QP Optimization, Inverse Kinematics, MyArm Robotic Arm, Talos Humanoid Robot)

Robotic Systems I

Konstantinos Asimakopoulos *k_asimakopoulos@ac.upatras.gr*
Lampros Printzios *printzios_lampros@ac.upatras.gr*

Department of Electrical and Computer Engineering
University of Patras

May 18, 2026

Table of Contents

Preliminaries - Forward Kinematics

Forward kinematics (FK) computes the end-effector (body frame) pose $\mathbf{T_b}$ from the robot joint variables $\mathbf{q}$, via a map f_{fk}.

$$\boxed{\mathbf{T_b} = f_{fk}(\mathbf{q})} \quad , \qquad \mathbf{q} \in \mathbb{R}^n, \ \mathbf{T_b} \in SE(3)$$

Two common formulations:

1. **Denavit–Hartenberg (DH) parameters**
   - Classical link-frame convention
   - Compact and widely used in introductory robotics
   - Depends on assigning frames carefully along the kinematic chain
2. **Product of Exponentials (PoE)**
   - Modern robotics formulation based on Lie Theory
   - Represents motion as a product of joint screw motions
   - Naturally expressed using $SE(3)$ Group and $\mathfrak{se}(3)$ Algebra

Preliminaries - Inverse Kinematics

Inverse kinematics (IK) computes the joint configuration $\mathbf{q}$ that produces a desired end-effector pose $\mathbf{T_b}$, via an inverse map $f_{ik} = f_{fk}^{-1}$.

$$\boxed{\mathbf{q} = f_{ik}(\mathbf{T_b}) = f_{fk}^{-1}(\mathbf{T_b})} \quad , \qquad \mathbf{T_b} \in SE(3),\ \mathbf{q} \in \mathbb{R}^n$$

IK is usually harder than FK because the **equations are nonlinear** and can present **singular or redundant solutions**.

Main ideas:

1. **Input**: desired position and orientation in task space
   Output: joint angles that realize that pose
2. May have **multiple**, **no**, or **infinitely many** solutions
3. Solved using **analytic methods** for closed-form solutions, usually based on trigonometry manipulations, or **numerical methods** for general robot structures, like Newton-Raphson, Levenberg-Marquardt, and damped least squares optimization

Preliminaries - Differential Kinematics

Differential kinematics relates joint velocities $\dot{\mathbf{q}}$ to the end-effector twist $\mathbf{V}$, through the robot Jacobian.

$$\boxed{\mathbf{V} = \frac{\partial f_{fk}(\mathbf{q})}{\partial \mathbf{q}} \dot{\mathbf{q}} = J(\mathbf{q})\, \dot{\mathbf{q}}} \quad , \qquad \mathbf{q} \in \mathbb{R}^n,\ \mathbf{V} \in \mathbb{R}^6,\ J \in \mathbb{R}^{6\times n}$$

where $J(\mathbf{q})$ is the **Jacobian matrix**. The Jacobian describes how small changes in joint variables affect the robot pose in Cartesian space.

Why it matters:

1. Used for **velocity control** and **motion tracking**
2. **Connects joint space to task space**
3. Helps **detect singularities**, where motion becomes constrained
4. Essential for resolved-rate control and **numerical IK**

Preliminaries - Linearization in Kinematics & Solving Linear Systems

For task-space control and IK, the key local approximation is:

$$\Delta x \approx J(q)\Delta q$$

This is the bridge between forward kinematics and iterative inverse kinematics.

In practice, most numerical IK methods repeatedly:

1. linearize the nonlinear kinematics around the current configuration,
2. solve a local linear or quadratic problem,
3. update the joints and repeat until the task error becomes small.

Solving IK - Newton–Raphson Method

Newton–Raphson is a root-finding method. Its idea is to solve a nonlinear equation by local linearization.

Define the IK error:

$$e(q) = x_d - f(q)$$

Linearize around the current iterate q_k:

$$e(q_k + \Delta q) \approx e(q_k) - J(q_k)\Delta q$$

Set the next error to zero:

$$e(q_k) - J(q_k)\Delta q = 0$$

Hence,

$$\Delta q = J(q_k)^{-1} e(q_k)$$

and the update is:

$$q_{k+1} = q_k + \Delta q$$

Fast convergence, but only locally and only when the Jacobian is square and invertible.

Solving IK - Newton-style Variants

Several Newton-style updates are used in robotics:

Exact Newton

$$\Delta q = J^{-1} e$$

Requires a square, nonsingular Jacobian.

Pseudoinverse Newton

$$\Delta q = J^{\dagger} e$$

Handles rectangular Jacobians and least-squares solutions.

Damped Newton / Levenberg–Marquardt

$$\Delta q = (J^T J + \lambda^2 I)^{-1} J^T e$$

Much more robust near singularities.

In robotics, the damped version is usually the practical default.

Solving IK - Damped Least Squares

Damped Least Squares, also known as Levenberg–Marquardt, improves robustness by penalizing large joint updates.

$$\min_{\Delta q} \|J\Delta q - e\|^2 + \lambda^2 \|\Delta q\|^2$$

The closed-form update is:

$$\Delta q = (J^T J + \lambda^2 I)^{-1} J^T e$$

This is the damped pseudoinverse:

$$J_\lambda^\dagger = (J^T J + \lambda^2 I)^{-1} J^T$$

Benefits:

- robust near singularities,
- smooth behavior,
- often the best default numerical IK method.

Solving IK - Quadratic Programming

Quadratic Programming (QP) is an optimization problem with a quadratic objective and linear constraints. The standard form is:

$$\min_x \ \frac{1}{2} x^T Q x + q^T x$$

subject to:

$$Ax = b, \qquad Cx \leq d$$

$$x_{min} \leq x \leq x_{max}$$

We focus on the convex case, where:

$$Q \succeq 0$$

QPs are central in robotics because they are fast and can encode constraints directly.

Solving IK - QP Cases

1. **Unconstrained case:**
$$x^* = -Q^{-1}q$$
when $Q \succ 0$.
2. **Equality-constrained case:** solved with KKT conditions.
3. **Bound-constrained case:**
$$x_{\min} \leq x \leq x_{\max}$$
common in control and robotics.
4. **Least squares:**
$$\min_x \|Ax - b\|^2$$
becomes a QP after expansion.

Solving IK - QP Solution Methods

Typical solution families for QPs include:

1. **Interior Point** method, with *Primal-Dual* algorithms.
2. **Active Set method**, that works well with *equality-constrained QPs* (inequalities may be converted to equalities using *slack variables*).
3. **Penalty methods**, like *Augmented Lagrangian* and *Barrier functions* (e.g. logarithmic barrier).
4. **Conjugate Gradient** method for *large QPs*.
5. **Gradient Projection** method, that works most efficiently for *bound-constrained QPs*.

In practice, robotics libraries often rely on *fast dedicated solvers* such as `qp_solvers`, `OSQP`, or `proxsuite`.

Solving IK - QP Optimization approach

We can reformulate IK as an optimization problem and add constraints directly. Let the desired task-space velocity be V_b and the current velocity be:

$$V = J_b(q)v$$

The least-squares tracking objective is:

$$\min_v \|J_b v - V_b\|^2$$

QP form:

$$Q = J_b^T J_b, \qquad q = -J_b^T V_b$$

This is the basic QP IK formulation.

Solving IK - Joint Limits as Linear Constraints

Joint limits can be enforced using velocity updates:

$$q_{k+1} = q_k + dt\, v$$

With box constraints:

$$q_{\min} \leq q_{k+1} \leq q_{\max}$$

This becomes:

$$q_k + dt\, v \leq q_{\max}, \qquad q_k + dt\, v \geq q_{\min}$$

Standard QP inequality form:

$$Cv \leq d$$

where

$$C = \begin{bmatrix} dt\, I \\ -dt\, I \end{bmatrix}, \qquad d = \begin{bmatrix} q_{\max} - q_k \\ q_k - q_{\min} \end{bmatrix}$$

This is the main reason QP is so useful in robotics.

Solving IK - Method Comparison

All the methods described solve the same **local IK problem**. They differ in how they handle *inversion*, *conditioning*, *redundancy*, and *constraints*.

Method	Speed	Robustness	Constraints	Singularities
Newton–Raphson	High	Low	No	Poor
Pseudoinverse Jacobian	High	Low–Medium	No	Poor
Damped Least Squares	High	High	No	Good
QP optimization	Medium–High	Very High	Yes	Excellent

Practical recommendation:

- use Newton or pseudoinverse for clean toy problems,
- use damped least squares for robust unconstrained IK,
- use QP when joint limits, priorities, or safety constraints matter.

QP Optimization-based Control for MyArm Robotic Arm Exercise - MyArm 300 Pi 7-DOF Robot

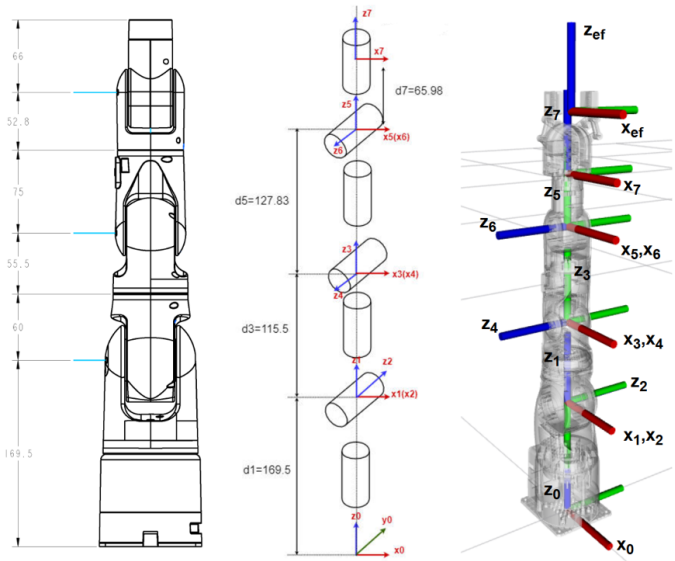
1. **MyArm 300 Pi** is a *7-DOF collaborative robotic arm* developed by Elephant Robotics.
2. Integrated **Raspberry Pi 4B** single-board dual-display computer with Ubuntu Mate 20.04.
3. Designed for robotics projects and research, with **ROS/ROS2 development**.
4. **Redundant kinematic structure** enables *human-arm-like dexterity* and *obstacle avoidance*.
5. Its **main specifications**: 200 gr payload capacity, 300 mm maximum reach, Ethernet / WLAN communication and ± 0.5 mm repeatability.

MyArm 300 Pi 7-DOF

QP Optimization-based Control for MyArm Robotic Arm Exercise - MyArm 300 Pi 7-DOF Robot

Denavit-Hartenberg Parameters (standard DH & modified MDH)

	a_i [m]		α_i [rad]		d_i [m]		θ_i [rad]	
Joint i	DH	MDH	DH	MDH	DH	MDH	DH	MDH
1	0	0	$-\pi/2$	0	0	0.1695	q_1	q_1
2	0	0	$\pi/2$	$-\pi/2$	0	0	q_2	q_2
3	0	0	$\pi/2$	$\pi/2$	0.1155	0.1155	q_3	q_3
4	0	0	$-\pi/2$	$\pi/2$	0	0	q_4	q_4
5	0	0	$\pi/2$	$-\pi/2$	0.1278	0.1278	q_5	q_5
6	0	0	$-\pi/2$	$\pi/2$	0	0	q_6	q_6
7	0	0	0	$-\pi/2$	0.0660	0.0660	q_7	q_7



MyArm kinematic structure in zero configuration (measurements in mm)

Space and Body Screw Axes $\begin{bmatrix}\omega & \mathbf{v}\end{bmatrix}^\top$ (S.I.)

Joint i	Space Screw S_i	Body Screw B_i
1	$(0, 0, 1, 0, 0, 0)$	$(0, 0, 1, 0, 0.0, 0)$
2	$(0, 1, 0, -0.1695, 0, 0)$	$(0, 1, 0, 0.3093, 0, 0)$
3	$(0, 0, 1, 0, 0, 0)$	$(0, 0, 1, 0, 0, 0)$
4	$(0, -1, 0, 0.285, 0, 0)$	$(0, -1, 0, -0.1938, 0, 0)$
5	$(0, 0, 1, 0, 0, 0)$	$(0, 0, 1, 0, 0, 0)$
6	$(0, -1, 0, 0.4128, 0, 0)$	$(0, -1, 0, -0.0660, 0, 0)$
7	$(0, 0, 1, 0, 0, 0)$	$(0, 0, 1, 0, 0, 0)$

Joint Limits

Joint	Minimum [°]	Maximum [°]
1	−160	160
2	−70	115
3	−170	170
4	−110	75
5	−170	170
6	−115	115
7	−180	180

QP Optimization-based Control for MyArm Robotic Arm
Exercise - MyArm IK: Problem Setup

Goal: move the MyArm end-effector to a desired target while respecting robot limits.

- **Position-only IK** for the end-effector position,
- **Full-pose IK** for both position and orientation.

In both cases, the solver follows these steps:

- linearize the kinematics,
- solve a damped least-squares QP,
- repeat until the error is small.

Why QP?

- regularization keeps the step stable,
- joint and velocity limits can be added directly,
- the same structure extends to larger optimization-based controllers.

What the assignment asks for

- formulate the cost matrix,
- formulate the linear term,
- build inequality constraints,
- create the QP object,
- test Newton, DLS, and QP behavior.

Robot context

- 7-DOF redundant arm,
- joint bounds matter,
- local Jacobian-based control is enough for the lab.

QP Optimization-based Control for MyArm Robotic Arm Exercise - Position-Only IK

The position task ignores orientation completely and only tracks the Cartesian position of the end-effector.

Linear body twist error

$$V_{b,p} = \frac{p_d - p(q)}{\Delta t}$$

Differential kinematics:

$$\dot{p} = J_p(q)\dot{q}$$

so the linearized tracking problem becomes:

$$\min_{\dot{q}} \ \frac{1}{2}\|J_p\dot{q} - V_{b,p}\|^2 + \frac{\lambda^2}{2}\|\dot{q}\|^2$$

This is the Damped Least Squares formulation used in the position solver.

QP form

$$\min_{\dot{q}} \ \frac{1}{2}\dot{q}^T Q\dot{q} + q_{lin}^T\dot{q}$$

with

$$Q = J_p^T J_p + \lambda^2 I, \qquad q_{lin} = -J_p^T V_{b,p}$$

Interpretation

- the Jacobian maps velocities to task-space motion,
- the damping avoids unstable steps,
- the solution is a regularized least-squares update.

QP Optimization-based Control for MyArm Robotic Arm Exercise - Constraints and Position Solver

The position solver also enforces joint position limits and joint velocity limits.

Joint limits

$$q_{k+1} = q_k + \Delta t\, \dot{q}$$

$$q_{lb} \leq q_k + \Delta t\, \dot{q} \leq q_{ub}$$

that become linear inequality constraints.

Velocity limits

$$\dot{q}_{lb} \leq \dot{q} \leq \dot{q}_{ub}$$

Combined form

$$C\dot{q} \leq d$$

This is exactly the structure that the QP solver expects.

In position IK solver

- compute `Jp`,
- compute `Vb_linear`,
- set `Q` and `q_lin`,
- assemble `C` and `d`,
- solve for `v_linear`.

Comparison with simpler IK

- Newton is very fast near a solution,
- DLS is more robust,
- QP inserts constraints naturally, without changing the local linearization idea.

QP Optimization-based Control for MyArm Robotic Arm Exercise - Full-Pose IK and Method Comparison

Full-pose IK uses the body twist error and the full Jacobian instead of only the translational part.

Full body twist error

$$V_b = \frac{\mathrm{Log}(T_d T^{-1})}{\Delta t}$$

Differential kinematics:

$$V_b = J(q)\dot{q}$$

Damped least-squares objective:

$$\min_{\dot{q}} \ \frac{1}{2}\|J\dot{q} - V_b\|^2 + \frac{\lambda^2}{2}\|\dot{q}\|^2$$

QP terms

$$Q = J^T J + \lambda^2 I, \qquad q_{lin} = -J^T V_b$$

In full pose IK solver

- use the full Jacobian `J`,
- use the full body twist error `Vb`,
- keep the same joint-limit constraints,
- solve iteratively until convergence.

Practical takeaway: Newton is clean but fragile, DLS is robust, and QP is the most flexible because it naturally handles bounds and can be extended later.

QP Optimization-based Control for Talos Humanoid Robot
Exercise - Talos Humanoid Robot Overview

Talos is a high-performance humanoid robot designed for research in walking, whole-body control, and manipulation.

Basic specifications

- Degrees of freedom: 32
- Height: 175 cm & Weight: 95 kg
- Payload per arm: 6 kg with the arm stretched
- Battery autonomy: about 1.5 h walking / 3 h stand-by
- Torque sensors: full feedback in all joints, except head, wrists, and grippers
- Software stack: Ubuntu LTS / real-time OS, fully developed in ROS

Talos humanoid robot

QP Optimization-based Control for Talos Humanoid Robot
Exercise - Talos Whole-Body IK: Tasks

The Talos exercise uses whole-body inverse kinematics for walking.

Simultaneous tasks

- left foot pose tracking,
- right foot pose tracking,
- center of mass tracking.

Each task contributes:

- a task-space error,
- a Jacobian.

The errors are stacked as:

$$e = \begin{bmatrix} e_{left\_foot} \\ e_{right\_foot} \\ e_{CoM} \end{bmatrix}$$

And the Jacobians are stacked as

$$J = \begin{bmatrix} J_{left\_foot} \\ J_{right\_foot} \\ J_{CoM} \end{bmatrix}$$

Why stacking matters

- it creates one optimization problem,
- the solver trades off tasks consistently,
- task weights give different priorities.

QP Optimization-based Control for Talos Humanoid Robot Exercise - Weighted QP

The whole-body objective is a weighted least-squares problem solved as a QP.

Weighted objective

$$\min_v \frac{1}{2}\|W(Jv - e)\|^2$$

which expands to:

$$\min_v \frac{1}{2} v^T Q v + q^T v$$

$$Q = (WJ)^T(WJ) + \lambda I, \quad q = -(WJ)^T We$$

Meaning of the weights

- large weights show priorities,
- feet are often prioritized over CoM,
- task priorities are encoded softly, not as hard rules.

Typical example weights

$$w_{feet} = 20, \qquad w_{CoM} = 5$$

or even larger when stabilizing a phase transition.

In the code

- stack task Jacobians and errors,
- horizontally stack weights,
- build `Q` and `q`,
- use a small regularizer,
- solve one QP per iteration.

QP Optimization-based Control for Talos Humanoid Robot Exercise - Constraints and Solver Loop

The QP enforces joint limits by constraining the integrated configuration update.

Constraint model

$$d_{min} \leq Cv \leq d_{max}$$

with

$$C = \Delta t I$$

The vectors `d_min` and `d_max` are computed from the current joint ranges.

Solver loop

- compute body errors and Jacobians,
- stack all tasks,
- build the weighted QP,
- solve for the generalized velocity,
- integrate the configuration,
- repeat until convergence.

Exercise code map

- stack errors and Jacobians,
- build `weights`,
- use $\lambda = 10^{-4}$,
- set `C`, `d_min`, `d_max`,
- run `test_walk.py`.

Interpretation

- feet stay stable,
- CoM shifts when needed,
- the robot advances through repeated QP solves.

References

- Newton-Raphson method, from wikipedia and Kevin M. Lynch (Modern Robotics)
- Augmented Lagrangian method (wikipedia's article)
- Line Search and Backtracking Line Search with Armijo rule (wikipedia's articles)
- Quadratic Programming Optimization (wikipedia's article)
- Modern Robotics: Mechanics, Planning, and Control; Kevin M. Lynch and Frank C. Park (Chapters 4-6, 10-11)
- Nocedal Optimization; Jorge Nocedal and Stephen J. Wright (Chapter 16, 17 for Quadratic Programming & Penalty and Augmented Lagrangian Methods)
- Convex Optimization; Stephen Boyd and Lieven Vandenberghe
- Product of Exponentials, from wikipedia and Kevin M. Lynch (Modern Robotics)
- MyArm 300 Pi 7-DOF Robotic Arm (Elephant Robotics Documentation)